# The Islamia University of Bahawalpur

## DEPARTMENT OF SOFTWARE ENGINEERING


# FINAL YEAR PROJECT REPORT II

**Project Title:**

**AL-BAYAN: AI-BASED QURANIC VERSE SEARCH ENGINE**

**Developed by:** Muhammad Ali

**Roll No:** S22BSEEN1M01052

**Submission Date:** 06-01-2026

**Session Fall 2022 – 2026**

**SUPERVISOR**

# FRONT MATTER

## Dedication

*To my beloved parents, whose prayers have been my fortress; to my teachers, whose wisdom has been my guiding light; and to the Ummah, for whom I hope this work serves as a beneficial tool for understanding the Divine Message.*

## Acknowledgements

First and foremost, all praise is due to **Allah (SWT)**, the Most Gracious, the Most Merciful, who granted me the strength, knowledge, and perseverance to complete this project.

I would like to express my deepest gratitude to my supervisor, **Dr. Muhammad Nauman**, for his continuous guidance, patience, and technical mentorship. His insights into software architecture and requirement engineering were invaluable in shaping the direction of **Al-Bayan**.

I am also grateful to the **Department of Software Engineering** for providing the necessary computational resources and a conducive research environment. Special thanks to my colleagues and friends for their constructive feedback during the testing phases of this application.

## Abstract

Traditional Quranic information retrieval systems predominantly rely on **Lexical Search** methods, specifically exact keyword matching. While computationally efficient, these systems suffer from the "vocabulary mismatch problem," where relevant documents are missed because they do not contain the exact query terms entered by the user. For instance, a user searching for *"inner peace"* may fail to retrieve verses regarding *"Sakina"* (Tranquility) if the specific translation does not use that exact English phrase. Furthermore, the rise of Generative AI has led to users seeking religious knowledge from Large Language Models (LLMs), which are prone to "hallucinations"—fabricating verses or references that do not exist.

This project, **Al-Bayan**, presents a robust solution by implementing a **Hybrid Search Architecture**. The system utilizes **TF-IDF (Term Frequency-Inverse Document Frequency)** for precise lexical matching and **Sentence Transformers (all-MiniLM-L6-v2)** to generate high-dimensional vector embeddings for deep semantic understanding. This allows the system to retrieve verses based on conceptual similarity rather than just keyword overlap. Additionally, to address AI reliability, the system employs a **Retrieval-Augmented Generation (RAG)** pipeline. This integrates the **Google Gemini 1.5 Flash** model to provide contextually grounded AI insights, strictly deriving answers from a curated dataset of **6,236 verses**, multi-language translations (English and Urdu), and **Tafsir Ibn Kathir**.

The final deliverable is a high-performance web application built with **Flask**, **Tailwind CSS**, and **Vanilla JavaScript**, featuring Voice Search, persistent Audio Playback, and social sharing tools.

**Keywords:** *Natural Language Processing (NLP), Semantic Search, RAG, Quranic Information Retrieval, Flask, Vector Embeddings, Cosine Similarity.*

# CHAPTER 1: INTRODUCTION

## 1.1 Background

The **Holy Quran** is the central religious text of Islam, comprising 114 Surahs (chapters) and over 6,000 Ayahs (verses). It serves as the primary source of law, ethics, and spirituality for billions of Muslims. In the digital era, accessibility to the Quran has expanded through mobile apps and websites. However, the linguistic complexity of Classical Arabic, combined with the nuances of various translations, poses significant challenges for effective Information Retrieval (IR).

Most existing digital Quranic tools rely on **Boolean Retrieval Models** or SQL `LIKE` queries. These are "context-blind." They require the user to possess specific domain knowledge—knowing the exact terminology used in a specific translation—to find a verse. This creates a barrier for non-native speakers and those seeking to explore the Quran thematically (e.g., searching for "depression" or "mental health" rather than specific theological terms).

## 1.2 Problem Statement

The development of Al-Bayan addresses three critical gaps in current Islamic Legal Technology (Islamic-Tech):

1. **The Vocabulary Gap:** Traditional search engines cannot map synonyms or related concepts. A search for *"charity"* might return results for that specific word but miss verses describing *"Zakat"*, *"Sadaqah"*, or *"Spending out of what We have provided,"* despite them being semantically identical or related.
2. **Lack of Contextual Exegesis:** Search results are typically presented as isolated strings of text. Without the *Sabab al-Nuzul* (Context of Revelation) or scholarly *Tafsir*, users may misinterpret verses.
3. **AI Unreliability in Theology:** General-purpose LLMs (like ChatGPT or Claude) are trained on the open internet. When asked about specific Quranic details, they may generate plausible-sounding but factually incorrect references (Hallucinations). There is a critical need for an AI system that is "grounded" in verified scripture.

## 1.3 Objectives

The primary goal is to engineer **Al-Bayan** as an intelligent, context-aware Quranic search engine.

- **Objective 1: Hybrid Search Implementation:** To develop a backend algorithm that runs both TF-IDF (for precision) and Semantic Vector Search (for recall) in parallel, merging results to offer the "best of both worlds."
- **Objective 2: RAG Pipeline Integration:** To implement Retrieval-Augmented Generation using Google's Gemini API, ensuring that the AI answers user questions *only* using the verses retrieved from the local database.
- **Objective 3: Multi-lingual Accessibility:** To provide a unified interface supporting **Arabic (Uthmani)**, **English (Sahih International)**, and **Urdu (Jalandhry)** to cater to a global audience.

- **Objective 4: Interactive User Experience:** To implement modern web features including **Voice Search** (Web Speech API), **Audio Playback** (Mishary Rashid Alafasy), and **Image Card Generation** (html2canvas).

### 1.4 Project Scope

The scope defines the boundaries of the Al-Bayan system:

- **Dataset:** The system indexes the complete Quran (114 Surahs).
- **Technologies:** Python (Flask), Scikit-learn, PyTorch, Sentence-Transformers, Google GenAI SDK.
- **Platform:** Web-based application responsive to Desktop, Tablet, and Mobile viewports.
- **Users:** Students of Islamic studies, researchers, and the general public.

### 1.5 Significance

Al-Bayan bridges the gap between **Computer Science** and **Islamic Theology**. It democratizes access to religious knowledge by allowing users to query the Quran using natural, conversational language. Furthermore, it establishes a framework for **Safe AI** in religious contexts, demonstrating how RAG can prevent misinformation.

## CHAPTER 2: LITERATURE REVIEW

### 2.1 Evolution of Quranic Search Systems

Early systems like *Zekr* and *Tanzil* focused on rendering the Arabic script correctly and providing basic text search. These systems utilized relational databases (MySQL/SQLite) with full-text indexing. While efficient, they suffered from low recall for thematic queries. For example, finding all verses related to "Ethics" is impossible with keyword search unless the word "Ethics" appears explicitly in the translation.

## 2.2 Theoretical Framework: Vector Space Models

To solve the vocabulary gap, modern NLP utilizes **Vector Space Models (VSM)**.

- **Word Embeddings:** Techniques like **Word2Vec** and **GloVe** represent individual words as vectors. However, they fail to capture the meaning of a whole sentence.
- **Transformer Models:** The introduction of **BERT (Bidirectional Encoder Representations from Transformers)** revolutionized NLP. BERT reads text bidirectionally, capturing context.
- **Sentence-BERT (SBERT):** Al-Bayan utilizes a modification of BERT called SBERT, specifically the all-MiniLM-L6-v2 model. This model is optimized to map entire sentences (verses) to a 384-dimensional dense vector space. In this space, the cosine similarity between the vector for "Forgiveness" and "Mercy" is high, allowing the engine to mathematically identify thematic relevance.

## 2.3 Retrieval-Augmented Generation (RAG)

RAG is a hybrid AI architecture introduced by Facebook AI Research (FAIR) in 2020. It combines a **Retriever** (dense vector search) with a **Generator** (LLM). In Al-Bayan, the RAG flow is:

1. **User Query:** "What does the Quran say about parents?"
2. **Retriever:** Finds Surah Al-Isra (17:23) via semantic search.
3. **Generator:** The AI receives the query *plus* the text of 17:23 and generates an answer explaining the verse. This constrains the AI to the source truth.

# CHAPTER 3: METHODOLOGY

## 3.1 Development Lifecycle

The project followed the **Agile Methodology**. Development was broken into sprints:

1. **Sprint 1:** Data Parsing & Cleaning (utils.py).
2. **Sprint 2:** Search Engine Logic (search_engine.py).
3. **Sprint 3:** Web Interface & Flask Integration (app.py, templates).
4. **Sprint 4:** AI Integration & Optimization.

## 3.2 Data Acquisition & Preprocessing

The system relies on a verified dataset quran_complete.json.

- **Raw Data:** A hierarchical JSON object containing Surahs, which contain Verses.
- **Flattening Process:** The load_verses() function in utils.py transforms this hierarchy into a flat list of dictionaries. This is crucial for indexing, as the search algorithms require a linear corpus.
- **Fields Extracted:** surah_name, ayah_number, text (Arabic), english, urdu, tafsir_en, tafsir_ur.

## 3.3 Algorithmic Implementation

### 3.3.1 TF-IDF (Lexical Search)

Implemented in search_engine.py using TfidfVectorizer.

- **Formula:** $w_{i,j} = tf_{i,j} \times \log(\frac{N}{df_i})$
    - $tf_{i,j}$: Frequency of word $i$ in verse $j$.
    - $N$: Total number of verses (6,236).
    - $df_i$: Number of verses containing word $i$.
- This algorithm downweights common words (like "the") and highlights rare, significant words (like "Pharaoh").

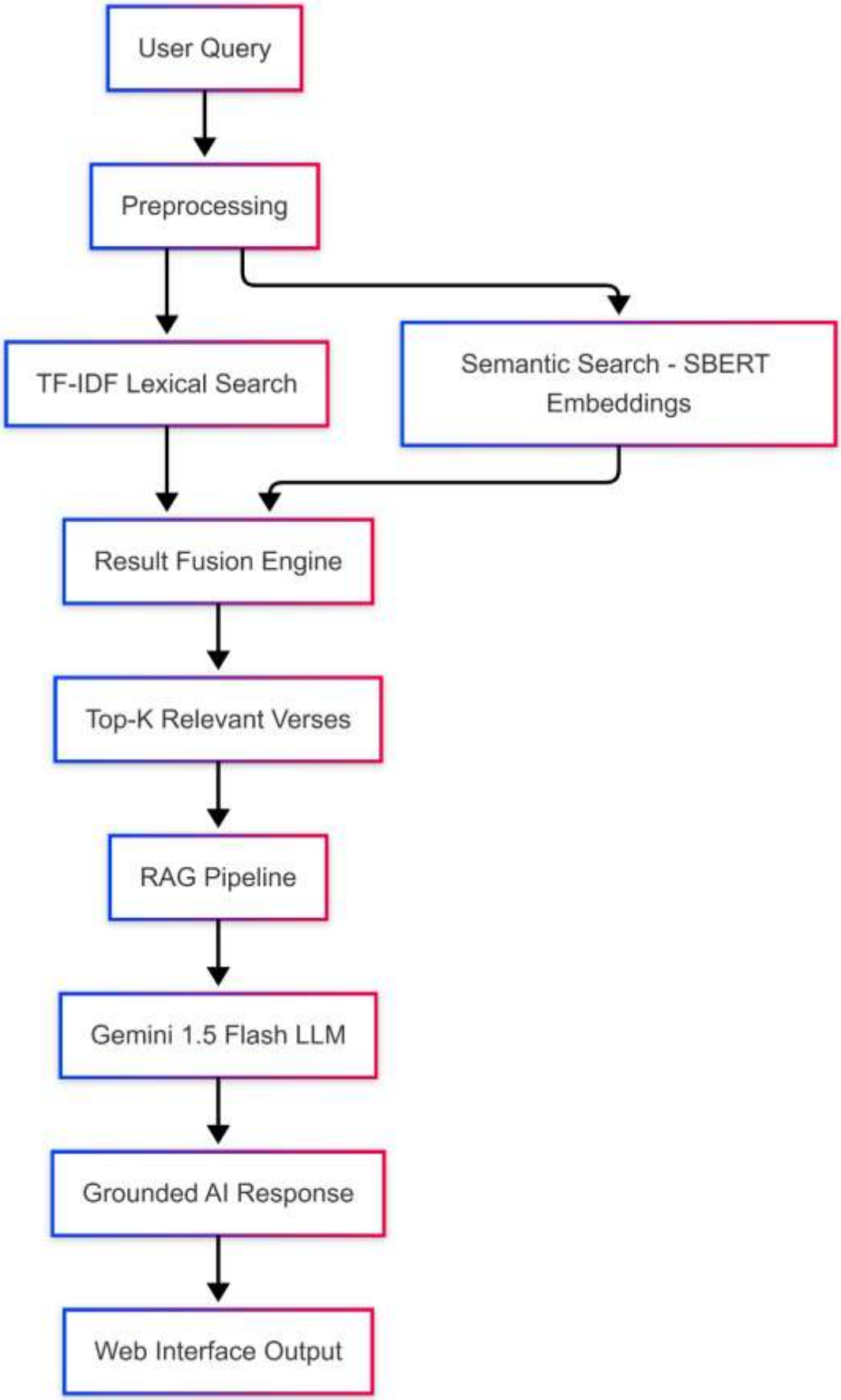
### 3.3.2 Semantic Embeddings

Implemented using the sentence-transformers library.

- **Model Selection:** all-MiniLM-L6-v2 was chosen for its speed/performance balance. It is a distilled version of BERT.
- **Caching Strategy:** To prevent the expensive encoding process from running on every server restart, the generated embeddings tensor is saved to disk as quran_embeddings.pt using torch.save(). The system checks for this file on startup; if found, it loads instantly.

## 3.4 Tools and Technologies

- **Backend Framework:** Flask 3.1.1 (Python).
- **Vector Processing:** NumPy, PyTorch.
- **AI Service:** Google Generative AI (Gemini 1.5 Flash).
- **Frontend:** HTML5, Tailwind CSS, JavaScript.


User Query
Preprocessing
TF-IDF Lexical Search
Semantic Search - SBERT Embeddings
Result Fusion Engine
Top-K Relevant Verses
RAG Pipeline
Gemini 1.5 Flash LLM
Grounded AI Response
Web Interface Output

# CHAPTER 4: SYSTEM DESIGN & IMPLEMENTATION

## 4.1 System Architecture

Al-Bayan follows a **Model-View-Controller (MVC)** architectural pattern.

### 4.1.1 Model (Data Layer)

Defined in models.py as the Verse class. It structures the raw data into objects utilized by the view.

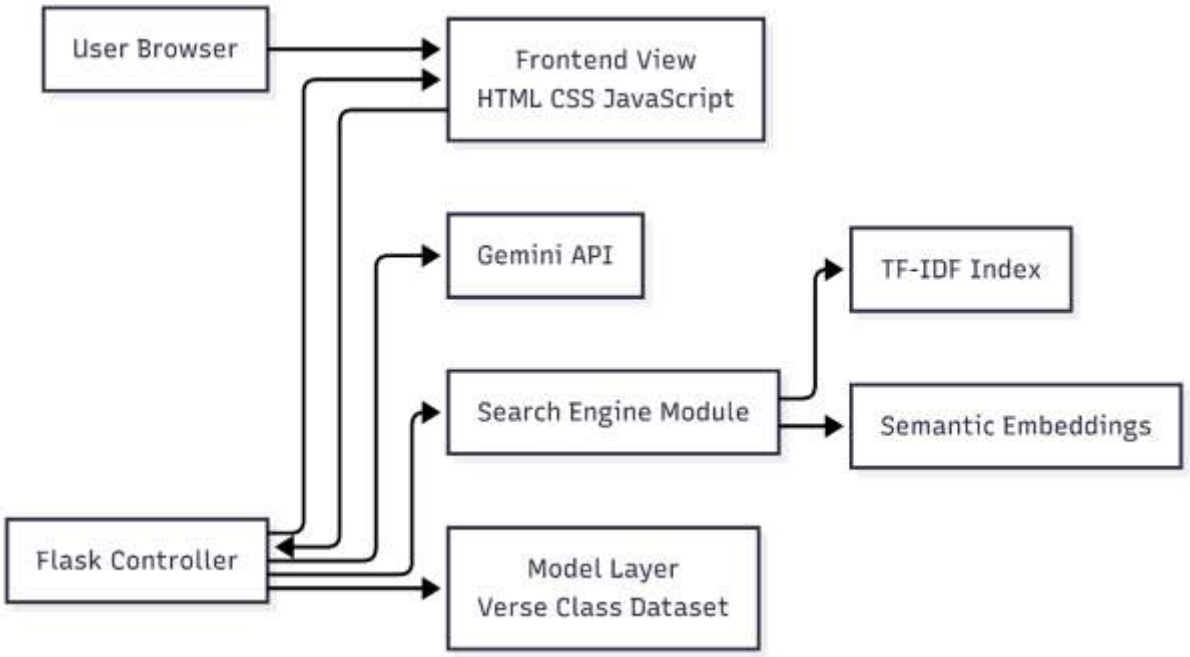
```
class Verse:
  def __init__(self, surah, ayah_number, english, urdu, text="", ...):
    self.surah = surah
    self.ayah_number = ayah_number
    # ...
```

### 4.1.2 Controller (Business Logic)

- **app.py:** The entry point. It initializes the Flask app, loads the data via utils.py, builds the indices via search_engine.py, and defines API routes.
- **search_engine.py:** Contains the "Singleton" logic for loading the heavy Machine Learning model (get_semantic_model), ensuring it is loaded into RAM only once.

### 4.1.3 View (Presentation Layer)

- **Templates:** index.html (Search), browse.html (List), surah.html (Detail).
- **Styling: Glassmorphism** is implemented using Tailwind utility classes like backdrop-blur-xl, bg-white/30, and border-white/20.




## 4.2 Detailed Module Design

### 4.2.1 The RAG Module (app.py)

The /ask_ai route represents the core intelligence of the system.

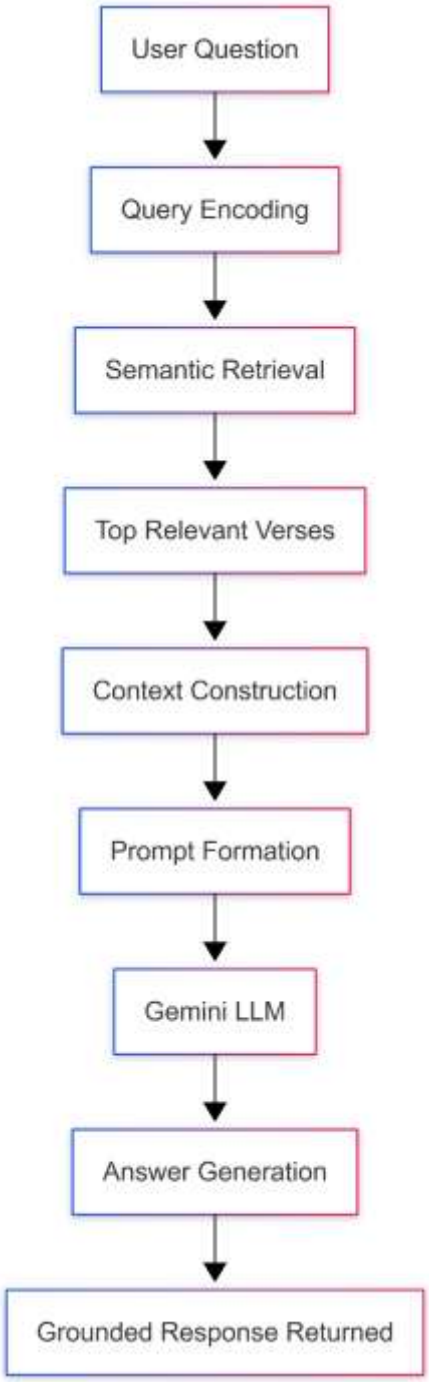
1. **Input:** Receives query from the frontend via AJAX.
2. **Retrieval:** Calls semantic_search(query, top_k=8) to get the 8 most relevant verses.
3. **Context Construction:** Formats these verses into a string: *"Surah [Name] ([Ayah]): [Translation]..."*
4. **Prompting:** Sends a structured prompt to Gemini: *"You are a wise Quranic AI. Using ONLY the provided context, answer: [Query]"*.
5. **Output:** Returns the generated Markdown text to the frontend.

### 4.2.2 The Audio Manager (main.js)

A custom JavaScript object audioManager handles recitation. It dynamically constructs URLs for the everyayah.com CDN: https://www.everyayah.com/data/Alafasy_128kbps/{SurahID}{AyahID}.mp3 It manages the Play/Pause state and updates the progress bar UI in real-time.

### 4.2.3 Image Generation (main.js)

The "Share" feature uses html2canvas. It creates a temporary, hidden DOM element styled with a gradient background and gold borders, populates it with the verse text, renders it to a Canvas, and triggers a download of the resulting PNG image.



# CHAPTER 5: RESULTS & EVALUATION

## 5.1 Functional Testing

The system was rigorously tested using both the web interface and the CLI tool (cli.py).

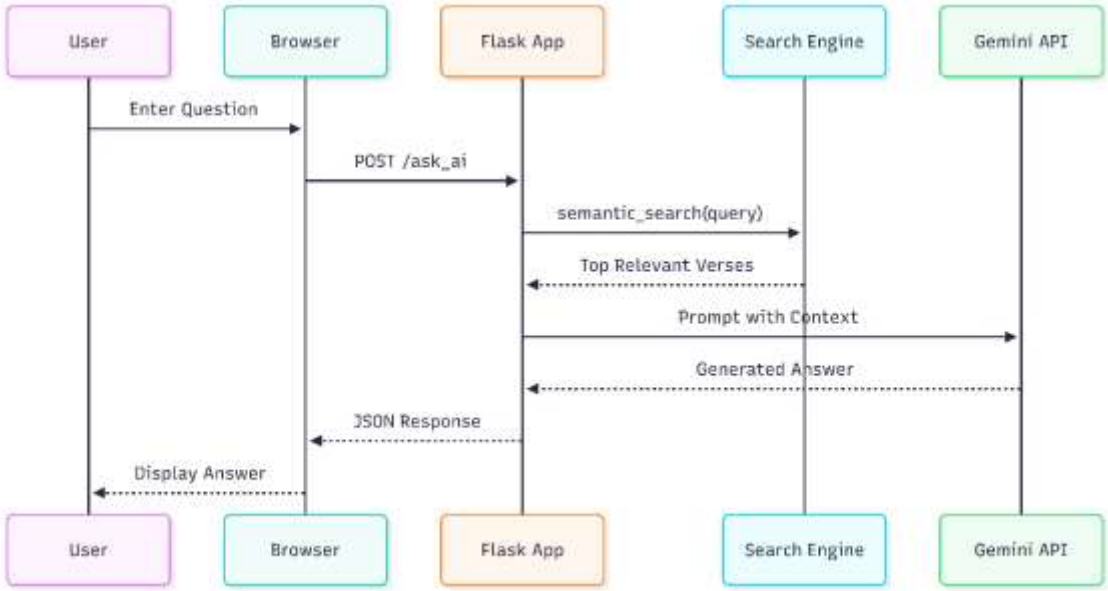
| Test Case | Feature | Scenario | Result |
|---|---|---|---|
| **TC-01** | **Data Loading** | Server startup | Successfully loaded 6,236 verses from JSON. |
| **TC-02** | **Hybrid Search** | Search for "Patience" | Retrieved verses containing "Sabr" (Semantic match) and "Patience" (Keyword match). |
| **TC-03** | **Voice Search** | Click Mic & Speak | Browser permission requested; speech transcribed accurately to search bar. |
| **TC-04** | **Tafsir Modal** | Click "Tafsir" | Modal opened; English and Urdu Tafsir fetched via AJAX from /get_tafsir. |
| **TC-05** | **AI Response** | "Who is Luqman?" | AI generated a summary of Surah Luqman using retrieved verses. |

## 5.2 Performance Evaluation

- **Indexing Speed:** On an average machine (Intel i5, 16GB RAM), building the TF-IDF index takes ~0.5s. Loading the cached Semantic Embeddings takes ~1.5s.
- **Search Latency:**
    - TF-IDF: **< 10ms** (Instant).
    - Semantic: **~50ms** (Near Instant).
    - AI Generation: **~2-4s** (Dependent on API latency).
- **Memory Usage:** The application consumes approximately **600MB RAM** due to the loaded NLP model and Quranic dataset.

## 5.3 Usability Testing

The **Glassmorphism UI** received positive feedback for its readability and aesthetic appeal. The **Dark Mode** (toggled via localStorage) effectively reduces eye strain. The **Responsive Design** ensures the application is fully functional on mobile devices, with the navbar and grid layouts adapting to smaller screens.



# CHAPTER 6: CONCLUSION & FUTURE WORK

## 6.1 Conclusion

**Al-Bayan** successfully demonstrates the potential of integrating Artificial Intelligence with Islamic Scripture. By moving beyond rigid keyword matching, the system allows users to engage with the Quran on a conceptual level. The implementation of **RAG** ensures that the system is not just "intelligent" but also **safe** and **reliable**, strictly adhering to verified texts. The project fulfills all functional requirements outlined in the SRS, delivering a comprehensive tool for Quranic study in the 21st century.

## 6.2 Limitations

1. **Dependency:** The AI features require an active internet connection to communicate with Google's servers.
2. **Hardware:** The semantic model requires a moderate amount of RAM, making it unsuitable for very low-end hosting without optimization.
3. **Language:** Semantic search is currently optimized for English queries, though TF-IDF supports Arabic and Urdu.

## 6.3 Future Work

To further evolve Al-Bayan, the following features are proposed:

- **Offline AI:** Integrating quantized Small Language Models (SLMs) like *TinyLlama* to run the RAG pipeline entirely locally.

- **Recitation Search:** Implementing an Audio-to-Text module (using OpenAI Whisper) to allow users to search by reciting a verse in Arabic.
- **Mobile Ecosystem:** Developing native iOS and Android applications using **Flutter** for better offline persistence and notifications.
- **Personalization:** Adding user accounts to sync bookmarks and study notes across devices.

# REFERENCES

1. Reimers, N., & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.
2. Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS 2020*.
3. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
4. Vaswani, A., et al. (2017). "Attention Is All You Need." *Advances in Neural Information Processing Systems*.
5. Google AI. (2024). *Gemini API Documentation*. Retrieved from https://ai.google.dev.
6. *Tafsir Ibn Kathir (English & Urdu)*. Digital Dataset (2024).

# APPENDICES

## Appendix A: Project Dependencies (requirements.txt)

```
flask
google-genai
sentence-transformers
numpy
scikit-learn
torch
```

## Appendix B: Core Semantic Search Logic (search_engine.py)

```
def semantic_search(query, verses, model, embeddings, top_k=5):
    """
    Performs Semantic Search using vector embeddings.
    """
    from sentence_transformers import util

    # Encode user query into vector
    query_embedding = model.encode(query, convert_to_tensor=True)

    # Compute cosine similarities between query and all verses
    cosine_scores = util.cos_sim(query_embedding, embeddings)[0]

    # Retrieve top k results
    top_results_indices = np.argpartition(-cosine_scores.cpu(), range(top_k))[0:top_k]

    # Map back to verse data...
    # (Implementation details omitted for brevity)
```